MATERIAL DOCENTE

UT00 PRESENTACION CURSO

Ejercicio 0 — «Tu primera aplicación web»

Práctica evaluable

Enunciado para el alumnado

Módulo profesional	Puesta en Producción Segura (5023)
Curso	Curso de Especialización en Ciberseguridad en Entornos de las TI
Normativa	RD 479/2020 (BOE) · Decreto 201/2021 (BOCM)
Stack vehicular	HTML5 · CSS3 · JavaScript (ES2023) · Node.js 22 LTS
Fecha	04/10/2026

Modalidad: individual · **Dedicación:** 3 h · **Obligatorio, no calificable numéricamente Entrega:** carpeta comprimida Apellidos_Nombre_EJ0.zip + una memoria breve en PDF

> **Qué se espera de ti.** Que construyas una aplicación web pequeña **que funcione**. Nada más. No se pide nada de seguridad: eso empieza en la unidad siguiente. Hoy se trata de recordar lo que ya sabes y de dejar el entorno montado.
>
> **No se califica con nota.** Se valora como **conseguido / en proceso / no conseguido**, y sirve para que el profesorado conozca el punto de partida real del grupo.
>
> 📌 **Guarda el proyecto cuando termines.** Lo volveremos a usar más adelante en el curso.

---

0. Antes de empezar: montar el entorno

Esto es la mitad del ejercicio y conviene hacerlo hoy, en clase, para resolver los problemas con ayuda.

0.1 · Qué terminal usar

Sistema	Qué abrir
Windows	**Git Bash** (viene con Git) o **WSL**. No uses PowerShell: algunos comandos del curso no funcionan ahí
macOS / Linux	La aplicación Terminal

0.2 · Instalar Node.js

1. Entra en nodejs.org y descarga la versión marcada como **LTS**.
2. Instálala con las opciones por defecto.
3. **Cierra la terminal y vuelve a abrirla** (si no, no encontrará el comando).

☑ Comprobación:

```
node -v
npm -v
```

Qué debe salir: una versión **22 o superior** (v22.x.x, v24.x.x…) y un número de versión de npm.

> **Si `node -v` no responde nada o da «command not found»:** la instalación no ha terminado o no has reabierto la terminal. Si persiste, dilo hoy en clase: no lo dejes para el día de la primera práctica.

0.3 · Un editor de código

Vale cualquiera. Si no tienes preferencia, Visual Studio Code es gratuito y es el más extendido.

---

1. Qué vas a construir

Una aplicación web mínima con sus **dos mitades**: lo que ve el usuario en el navegador y el servidor que responde.

Elige uno de estos tres temas —el que te apetezca— y úsalo durante todo el ejercicio:

Opción	La aplicación guarda…	Y permite buscar por…
A	Una lista de **películas** (título, director, año)	el título
B	Una lista de **contactos** (nombre, teléfono, ciudad)	el nombre
C	Una lista de **incidencias** (código, descripción, estado)	la descripción

En las instrucciones usaré la opción A como ejemplo. Si eliges otra, cambia los nombres de los campos y ya está.

Cómo va a quedar

```
mi-primera-web/
├── package.json          <- ficha del proyecto (se crea sola)
├── node_modules/         <- librerías descargadas (no se toca, no se entrega)
├── public/               <- LO QUE SE ENVÍA AL NAVEGADOR
│   ├── index.html
│   ├── estilo.css
│   └── app.js
└── src/
    └── servidor.js       <- LO QUE SE QUEDA EN EL SERVIDOR
```

---

2. Paso a paso

Paso 2.1 · Crear el proyecto

Escribe **línea a línea**, pulsando Intro después de cada una:

```
mkdir mi-primera-web
cd mi-primera-web
npm init -y
npm pkg set type=module
npm install express@4.19.2
mkdir public src
```

☑ **Comprobación:** ls debe mostrar node_modules, package.json, public y src.

> **Qué hace cada línea:** la primera crea la carpeta; `npm init -y` crea la ficha del proyecto; `npm pkg set type=module` permite escribir `import` en lugar de `require`; `npm install express` descarga la librería con la que se hacen servidores web.

Paso 2.2 · El servidor

Crea **src/servidor.js** con este contenido. Está comentado línea a línea: **léelo antes de copiarlo**.

```
// 1. Cargamos la libreria con la que se hacen servidores web
import express from "express";

// 2. Creamos la aplicacion
const app = express();

// 3. Todo lo que haya en la carpeta "public" se envia al navegador tal cual
app.use(express.static("public"));

// 4. Los datos. En una aplicacion real estarian en una base de datos;
//    aqui los dejamos en una lista para no complicar el ejercicio.
const PELICULAS = [
  { titulo: "El tercer hombre", director: "Carol Reed", anio: 1949 },
  { titulo: "Ladrones de bicicletas", director: "Vittorio De Sica", anio: 1948 },
  { titulo: "Rashomon", director: "Akira Kurosawa", anio: 1950 },
  { titulo: "Cantando bajo la lluvia", director: "Stanley Donen", anio: 1952 },
];

// 5. Que responder cuando el navegador pida /api/peliculas
app.get("/api/peliculas", (peticion, respuesta) => {
  // 5.1 Recogemos lo que ha escrito el usuario
  const busqueda = peticion.query.titulo;

  // 5.2 Nos quedamos con las peliculas cuyo titulo contenga ese texto
  const resultado = PELICULAS.filter((p) =>
    p.titulo.toLowerCase().includes(String(busqueda).toLowerCase())
  );

  // 5.3 Respondemos con los datos en formato JSON
  respuesta.json(resultado);
});

// 6. Arrancamos el servidor en el puerto 3000
app.listen(3000, () => console.log("Servidor en http://localhost:3000"));
```

Paso 2.3 · Arrancarlo y comprobar que responde

```
node src/servidor.js
```

Qué debe salir: Servidor en http://localhost:3000

Deja esa terminal abierta: mientras lo esté, el servidor funciona. Para pararlo, Ctrl+C.

Ahora, en el navegador, entra en esta dirección:

```
http://localhost:3000/api/peliculas?titulo=la
```

Qué debe salir: las **dos** películas cuyo título contiene «la», todo en una línea:

```
[{"titulo":"Ladrones de bicicletas","director":"Vittorio De Sica","anio":1948},{"titulo":"Ca
ntando bajo la lluvia","director":"Stanley Donen","anio":1952}]
```

Aparece «Ladrones de bicicletas» porque **«La» está dentro de «Ladrones»**: la búsqueda mira si el texto está contenido en el título, no si es una palabra suelta. Si pruebas con `titulo=zzz` obtendrás `[]`, que significa «ninguna coincidencia».

📸 Captura 1.

Si dice `EADDRINUSE`: el puerto 3000 ya está ocupado por otro programa. Ciérralo, o cambia el `3000` por `3001` en la última línea y usa esa dirección.



Paso 2.4 · La página

Crea **public/index.html**. Esta parte **la escribes tú**: te doy la estructura mínima y tú la completas.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <title>Mi catálogo de películas</title>
  <link rel="stylesheet" href="estilo.css">
</head>
<body>
  <header>
    <h1>Catálogo de películas</h1>
  </header>

  <main>
    <!-- 1. El formulario de búsqueda. Necesita:
            - un <form> con un id
            - un <input> de texto con un id, para escribir el título
            - un <button> para enviar                                  -->

    <!-- 2. Una caja vacía donde aparecerán los resultados.
            Un <section> o un <div> con un id es suficiente.            -->
  </main>

  <script src="app.js" defer></script>
</body>
</html>
```

> **Qué significa `defer`:** que el navegador ejecute ese programa **cuando la página esté montada**. Sin él, el programa intentaría buscar el formulario antes de que exista.

Paso 2.5 · Los estilos

Crea **public/estilo.css**. Lo que quieras, pero **que se note que lo has tocado**: como mínimo, tipografía, color de fondo de la cabecera y algún margen.

```css
/* Un punto de partida; a partir de aqui, a tu gusto */
body {
  font-family: system-ui, sans-serif;
  margin: 0;
}

header {
  background: #14375e;
  color: #fff;
  padding: 1rem 2rem;
}

main {
  max-width: 900px;
  margin: 2rem auto;
  padding: 0 1rem;
}
```

Paso 2.6 · El comportamiento

Crea **public/app.js**. Esta es la parte con más trabajo. Tiene que hacer cuatro cosas:

```js
// 1. Localizar el formulario y la caja de resultados de la pagina
//    Pista: document.querySelector("#el-id-que-le-pusiste")

// 2. Decirle al formulario: "cuando alguien te envie, haz esto"
//    Pista: formulario.addEventListener("submit", async (evento) => { ... })
//    Y dentro, lo PRIMERO: evento.preventDefault();
//    (si no, el navegador recarga la pagina y no veras nada)

// 3. Leer lo que ha escrito el usuario y pedirselo al servidor
//    Pista: const respuesta = await fetch("/api/peliculas?titulo=" + loQueEscribio);
//           const datos = await respuesta.json();

// 4. Mostrar los resultados en la caja
//    Aqui hay varias formas de hacerlo. Usa la que conozcas o la que encuentres.
```

> **Sobre el punto 4:** hay más de una manera de meter resultados en una página y **todas valen para este ejercicio**. Usa la que recuerdes del curso pasado o la que encuentres buscando. No hay respuesta «correcta» que debas adivinar.

Paso 2.7 · Probarlo entero

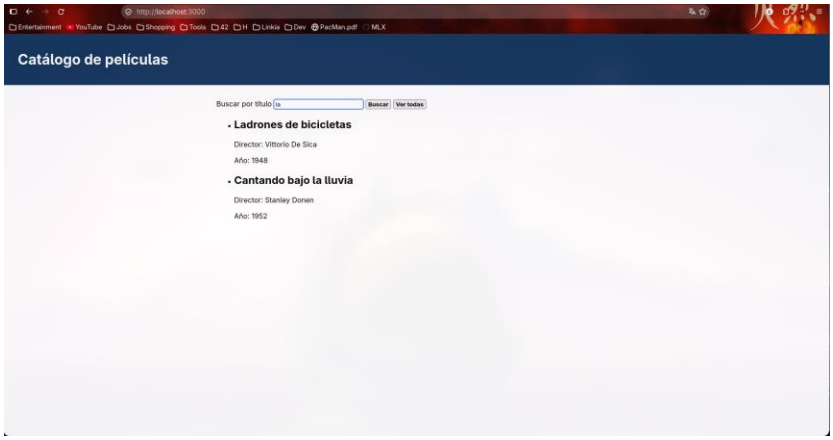
1. Asegúrate de que el servidor sigue arrancado (node src/servidor.js).
2. Abre http://localhost:3000 en el navegador.

3. Escribe `la` en el buscador y pulsa el botón.

Qué debe pasar: aparecen las películas que contienen «la» en el título, en la caja de resultados.

📸 **Captura 2:** la página con resultados en pantalla.

> **Si no pasa nada al pulsar el botón:** abre las herramientas del navegador con **F12**, ve a la pestaña **Consola** y mira si hay algún mensaje en rojo. Casi siempre dice exactamente qué falta.



3. Las dos preguntas

Cuando funcione, responde en la memoria. **No busques en internet: se trata de lo que tú creas ahora.** Volveremos a ellas más adelante en el curso.

Pregunta 1 · ¿Dónde se ejecuta cada cosa?

Fichero	¿Qué hace? (una línea)	¿Se ejecuta en el navegador o en el servidor?
index.html	Guarda la estructura de la página web	Navegador
estilo.css	Guarda los estilos de la página web	Navegador
app.js	Se encarga de recibir y responder peticiones que recibe la aplicación del cliente	Servidor
servidor.js	Se encarga de recibir y responder peticiones que recibe el servidor del cliente	Servidor

Pregunta 2 · Tres líneas

> **Si un usuario escribe algo raro en el buscador —un texto larguísimo, símbolos, código—, ¿qué crees que pasaría?**

No hay respuesta correcta todavía. Escribe lo que te parezca y **guárdalo tal cual**: no lo cambies después.

En primera instancia, el usuario no recibirá la respuesta para el que el servidor fue construido. Pero, aún sin gran conocimiento del campo, el servidor podría recibir ataques de inyecciones, pues en el back-end de la página web no se tiene constancia de estos casos.

4. Si quieres más (opcional)

Solo si has terminado y te apetece. **No hace falta para dar el ejercicio por conseguido.**

- **Añadir un dato más** a cada película y mostrarlo también en los resultados.
- **Mensaje de «sin resultados»** cuando la búsqueda no encuentre nada.
- **Un segundo criterio** de búsqueda: por director además de por título.
- **Ordenar** los resultados por año.

---

5. Qué hay que entregar

Cód.	Entregable	Formato
E1	El proyecto completo, **sin la carpeta node_modules**	ZIP
E2	Las dos capturas, con un pie que explique qué se ve	En la memoria
E3	Las respuestas a las dos preguntas del apartado 3	En la memoria
E4	Memoria breve, **2 o 3 páginas**, en PDF	PDF

Para borrar `node_modules` antes de comprimir:

```
rm -rf node_modules
```

> Pesa cientos de megas y se vuelve a descargar con `npm install` en diez segundos. **No se entrega nunca.**

Antes de entregar, comprueba que:

- ☐ El servidor arranca con `node src/servidor.js` sin errores
- ☐ La página se ve en http://localhost:3000
- ☐ Al buscar aparecen resultados

- ☐ Están los **cinco** ficheros: index.html, estilo.css, app.js, servidor.js y package.json
- ☐ Has borrado node_modules
- ☐ **Tienes una copia del proyecto guardada para ti**

---

6. Cómo se valora

Este ejercicio **no lleva nota**. Se valora así:

Nivel	Qué significa
Conseguido	La aplicación funciona de principio a fin y la memoria está completa
En proceso	Funciona a medias, o falta parte de la memoria. Se comenta individualmente y se recupera
No conseguido	No se ha entregado, o no llega a arrancar

Lo que de verdad se mira no es la elegancia del código, sino:

- que el entorno esté montado y funcionando,
- que sepas explicar **qué hace cada fichero y dónde se ejecuta**,
- y que hayas respondido la pregunta 2 con lo que piensas ahora, sin buscarlo.

Que te quede claro: aquí nadie espera código perfecto. Se espera código que funcione, escrito por ti, y que puedas explicar.

---

7. Ayuda rápida

Problema	Qué significa	Qué hacer
node: command not found	Node no está instalado o no reabriste la terminal	Repite el apartado 0.2
Cannot use import statement outside a module	Falta una línea en package.json	npm pkg set type=module
EADDRINUSE: port 3000	El puerto está ocupado	Cierra el otro programa o usa el 3001
Cannot find module 'express'	No se instaló la librería	npm install express@4.19.2
La página sale en blanco	El HTML no está en public/	Comprueba la carpeta y el nombre del fichero
Al pulsar el botón se recarga la página	Falta evento.preventDefault()	Paso 2.6, punto 2
No pasa nada al pulsar	Algún identificador no coincide	**F12** → **Consola**: el mensaje en rojo lo dice
Los estilos no se aplican	La ruta del CSS no es correcta	Debe ser href="estilo.css", sin carpeta delante

8. Recursos

- MDN Web Docs en español: *formularios*, *addEventListener*, *fetch*, *querySelector*.
- Documentación de Express: *Hello world* y *Serving static files*.
- Y lo más útil: **la pestaña Consola de las herramientas del navegador (F12)**. Casi todos los problemas de este ejercicio se diagnostican ahí.